



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

File Compression Tool Using Huffman Coding

Version 1.0 approved

Prepared by Debaditya Chakrabarti, Divisha Gupta, Greeshma N, Chandrakanth V G

PES University

09 September 2026



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, acronyms, and abbreviations

1.4 References

1.5 Overview

2. Overall description

2.1 Product perspective

2.2 Product functions

2.3 User characteristics

2.4 Constraints

2.5 Assumptions and dependencies

3. Specific requirements

3.1 External Interface

3.2 Functional Requirements

3.3 Non-Functional Requirements

3.4 Design Constraints

Appendixes

Index



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Divisha Gupta	09-09-2026	Initial SRS prepared for the Huffman-based file compression tool.	1.0

Introduction

Purpose

The product specified in this document is the File Compression Tool Using Huffman Coding, Version 1.0. It is a C/C++ command-line application that performs lossless compression and decompression of files using Huffman coding. This SRS covers the complete basic application, including byte-frequency analysis, Huffman-tree construction, code generation, bit packing, metadata storage, decompression, file recovery, statistics and error handling.

Intended Audience

This SRS is intended for the student developers, project guide, testers and users of the File Compression Tool. Developers can use it to understand the expected behaviour and implementation boundaries. Testers can use it to prepare test cases for compression, decompression and error handling. The document first introduces the product, then describes its overall context and functions, followed by interface requirements, system features, non-functional requirements, constraints and appendices.

Product Scope

The File Compression Tool reduces the storage space required by a file while preserving its contents. It assigns shorter binary codes to frequently occurring bytes and longer codes to less frequent bytes by using a Huffman tree. The user can compress a file into a custom compressed format and later decompress it to recover the original byte sequence. The main objectives are correct lossless recovery, a simple command-line workflow, understandable implementation of data structures and useful compression statistics.

References

The following references are used for this SRS:

1. Project Guidelines supplied for the course project, 2026.
2. SRS-Document template supplied for the course project.
3. Assigned problem statement: File Compression Tool (Basic ZIP Implementation), Project Number 14.
4. Course notes on Huffman coding, binary trees, priority queues and file handling.

Overall Description

Product Perspective

This is a new, self-contained academic software project. It does not depend on an existing database or online service. The user interacts with the command-line application, which reads an input file from the local file system. During compression, the Huffman module produces a tree and codes, the compression module writes metadata and packed bits, and the output is saved as a custom compressed file. During decompression, the stored metadata is used to rebuild the tree and recover the original file.

Product Functions

The major product functions are:

- Accept a file path from the user.
- Count the frequency of every byte in the input.
- Build a Huffman tree and generate prefix-free codes.
- Encode and pack the file contents into binary data.
- Store metadata required for decompression.
- Read a compressed file and reconstruct its Huffman tree.
- Decode the data and recreate the original file.
- Display file sizes, compression ratio and space saved.
- Report invalid files and other input or output errors.

User Classes and Characteristics

The system has one main user class: the end user. The end user has basic computer knowledge and can provide file paths and choose an operation from the menu. The user does not need to understand the Huffman algorithm. The student developers and testers are secondary users of the system because they run the tool while implementing and verifying its functions. No administrator account, privilege system or database user class is required.

Operating Environment

The application will operate on a personal computer running Windows or Linux with a standard C++ compiler. The planned implementation will use C++17 or a compatible version of standard C++. It will use the local file system and standard C++ libraries such as `iostream`, `fstream`, `vector`, `queue`, `string` and `filesystem` where available. The first interface will be command-line based and will not require an internet connection, server, database or browser.

Design and Implementation Constraints

The application shall be implemented in C/C++ using standard language features and data structures. The initial version shall be a basic ZIP-like implementation and is not required to be compatible with commercial ZIP, WinZip or 7-Zip files. Input and output shall be handled in binary mode so that text and binary files can be processed. The compressed-file format, including its header and metadata, must be defined by the team and used consistently by both compression and decompression. The program shall avoid losing the original file and shall report errors instead of silently overwriting an existing output file.

2.6 Assumptions and Dependencies

The project assumes that the user has read and write permission for the selected files and output folder. It assumes that the input file remains unchanged while compression is running. It also assumes that the C++ compiler and standard library are available on the system. The project depends on the correctness of the Huffman tree, the agreed compressed-file format and sufficient disk space for the output. No external service, database or third-party online component is required.

External Interface Requirements

User Interfaces

The user interface will be a command-line menu with options such as Compress File, Decompress File and Exit. The program will request an input path and, where required, an output path. It will display a success message after completion and show relevant statistics. Invalid paths, unsupported operations, malformed compressed files and output errors shall be displayed as clear error messages. The interface shall not require a graphical window or network connection.

Software Interfaces

The application has no mandatory external software interface. It uses the operating system file system through standard C++ file streams. Its internal modules communicate through data structures representing byte frequencies, Huffman nodes, codes, headers and encoded data. No database, cloud storage, external API or commercial compression library will be used.

Communications Interfaces

No communications interface is required. The system is designed to work locally and does not send file contents over a network. HTTP, FTP, e-mail and browser communication are therefore not applicable to Version 1.0.

No special hardware interface is required. The application uses the computer processor, memory and local storage through the operating system. A keyboard and display are sufficient for the command-line interaction.

Analysis Models

The main analysis model is a UML use-case model with one primary actor, the End User. The End User initiates file compression, file decompression and exit operations. Compression includes byte-frequency analysis, Huffman-tree construction, code generation, bit packing, compressed-file creation and display of compression statistics. Decompression includes metadata validation, Huffman-tree reconstruction, bit decoding and recovered-file writing. Figure 1 shows the required use-case model. An entity-relationship diagram is not applicable because the system does not use a database.

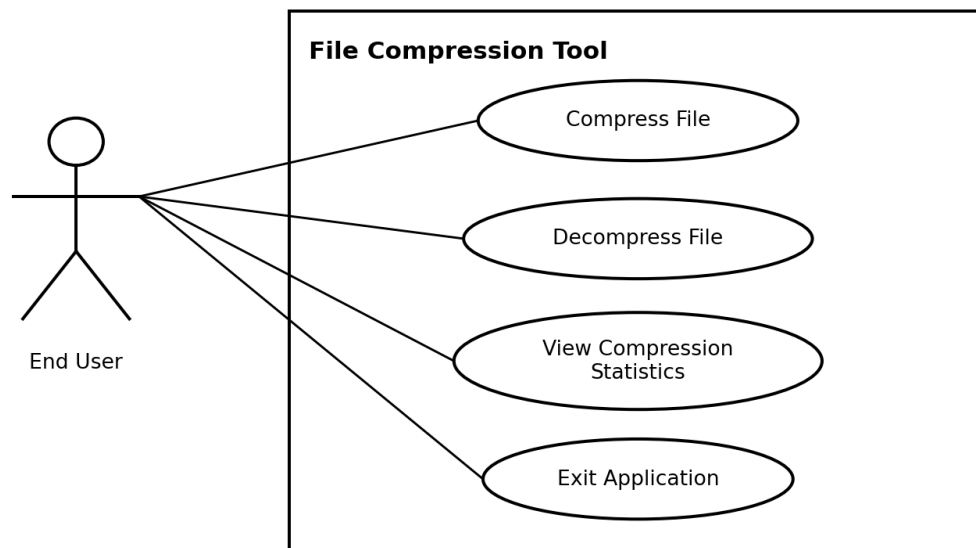


Figure 1. UML Use Case Diagram - File Compression Tool

System Features

The system is organised around the following features: file compression, file decompression, statistics display and error handling. These features are described in the following sections along with their priorities, stimulus/response sequences and functional requirements.

File Compression

This high-priority feature allows the user to select a normal file and create a compressed file using Huffman coding.

5.1.1 Description and Priority

Priority: High. The feature reads the input bytes, builds the frequency table and Huffman tree, generates codes, packs the encoded bits and writes the metadata and compressed data to an output file.

5.1.2 Stimulus/Response Sequences

1. The user chooses Compress File.
2. The system asks for the input path and output path.
3. The system checks that the input can be opened.

4. The system counts byte frequencies and builds the Huffman tree.
5. The system encodes and writes the compressed file.
6. The system displays success and compression statistics.
7. If a path or output operation is invalid, the system displays an error and returns to the menu.

5.1.3 Functional Requirements

REQ-1: The system shall accept a valid input file path for compression.

REQ-2: The system shall read the selected input file in binary mode without modifying the original file.

REQ-3: The system shall calculate the frequency of each byte value present in the input file.

REQ-4: The system shall construct a Huffman tree from the calculated byte frequencies.

REQ-5: The system shall generate a prefix-free Huffman code for every byte value present in the input file.

REQ-6: The system shall encode the input bytes using the generated Huffman codes and pack the encoded bits into bytes for storage.

REQ-7: The system shall store sufficient header metadata to reconstruct the Huffman tree and determine the valid encoded bit count during decompression.

REQ-8: The system shall write the header metadata and packed encoded data to a separate compressed output file.

REQ-9: After successful compression, the system shall display the original file size, compressed file size, compression ratio and percentage of space saved.

Each requirement in this SRS uses a unique identifier and is written so that it can be verified by executing the application and comparing the observed result with the specified behaviour.

File Decompression and Recovery

This high-priority feature allows the user to restore the original file from a compressed file created by the application.

5.2.1 Description and Priority

Priority: High. The feature reads and validates the header, reconstructs the Huffman tree, decodes the packed bits and writes the recovered bytes to a separate output file.

5.2.2 Stimulus/Response Sequences

1. The user chooses Decompress File.
2. The system asks for the compressed-file path and output path.
3. The system reads and validates the header and metadata.
4. The system rebuilds the Huffman tree.
5. The system reads the valid encoded bits and decodes them.
6. The system writes the recovered file and displays its status.

7. If the file is invalid, truncated or inaccessible, the system displays an error and does not create an incorrect recovered file.

5.2.3 Functional Requirements

REQ-10: The system shall accept a valid compressed-file path for decompression.

REQ-11: The system shall validate the compressed-file identifier, header and metadata before decoding the encoded data.

REQ-12: The system shall reconstruct the Huffman tree from the metadata stored in the compressed file.

REQ-13: The system shall decode only the valid packed bits using the reconstructed Huffman tree.

REQ-14: The system shall write the recovered bytes to the user-selected output file.

REQ-15: For a valid compressed file created by this application, the recovered file shall be byte-for-byte identical to the original file.

REQ-16: If the compressed file is invalid, corrupted, truncated or inaccessible, the system shall report an error and shall not report a successful recovery.

Other Nonfunctional Requirements

Performance Requirements

NFR-PERF-1: Frequency counting shall require one complete pass over the input file, and encoding shall require no more than one additional complete pass over the input file. The main file-processing work shall therefore grow linearly with the input size, excluding Huffman-tree construction over the fixed byte alphabet.

NFR-PERF-2: Every compression or decompression request shall terminate with either a completion message or a clear error message.

Safety Requirements

NFR-SAFE-1: The system shall not delete or modify the original input file during compression.

NFR-SAFE-2: Recovered data shall be written to a separate output file. If the selected output path already exists, the system shall ask for confirmation or report the conflict before writing.

NFR-SAFE-3: Malformed metadata and truncated compressed data shall be detected where possible and shall not cause the application to terminate abnormally.

Security

The application is a local file-compression utility and does not provide user accounts, encryption or network communication. The security focus for Version 1.0 is protection of local file confidentiality, file integrity and authorised file access.

Security Objectives

SO-1 - Confidentiality: Keep the contents of input, compressed and recovered files on the local system and prevent application-level transmission of file contents over a network.

SO-2 - Integrity and authorised access: Preserve the original file and ensure that the application performs file operations only through permissions granted by the operating system.

Security Requirements

SEC-REQ-1: The application shall perform compression and decompression locally and shall not establish a network connection or transmit file contents as part of either operation.

SEC-REQ-2: The application shall use the operating system file permissions for all reads and writes. If access is denied, the operation shall stop and display an error without modifying the input file.

SEC-REQ-3: The application shall validate the compressed-file identifier and required metadata before decoding and shall reject malformed input instead of reporting a successful recovery.

Huffman coding provides compression only and shall not be presented as encryption or confidentiality protection.

Software Quality Attributes

Correctness and reliability are the most important quality attributes. After compression followed by decompression, the recovered file shall match the original byte-for-byte. The command-line workflow shall be understandable to a user with basic computer knowledge. The code shall be modular and maintainable, with separate responsibilities for tree construction, compression, decompression and user interaction. The program shall handle empty files, one-symbol files and files containing all byte values.

Business Rules

The user must select a valid operation before a file is processed. Compression shall produce a file that can be read by the decompression operation of the same application. Decompression shall not be attempted on a file with an invalid header or unsupported format. The original file shall remain unchanged. The application shall preserve all byte values, including whitespace and non-printable binary values.

Domain requirements: Huffman codes shall be prefix-free so that encoded data can be decoded without ambiguity. The compressed format shall store enough information to identify the format and recover the original byte count or valid padding boundary. Lossless compression shall be used; the system shall not change, omit or approximate file data.

Other Requirements

The project shall be implemented as a basic local file-compression utility. It shall not require a database, user registration, internet access, encryption, cloud storage or compatibility with commercial ZIP archives in Version 1.0. The program output shall use clear units such as bytes and percentages for statistics. The project documentation shall include test cases for normal, empty, repetitive, binary and invalid inputs.

Appendix A: Glossary

Huffman coding: A lossless coding method that assigns shorter codes to frequent symbols and longer codes to less frequent symbols.

Byte: A unit of eight bits and the basic value processed by this tool.

Huffman tree: A binary tree used to generate Huffman codes.

Priority queue: A data structure that returns the item with the smallest frequency first during tree construction.

Bit packing: Combining individual encoded bits into full bytes for storage.

Metadata/Header: Information stored in the compressed file so that it can be decoded.
 Lossless compression: Compression in which decompression reproduces the exact original data.
 Compression ratio: Compressed size divided by original size, expressed as a percentage.
 Padding: Extra bits added to complete the final stored byte.

Appendix B: Field Layouts

The application has no database fields. The following appendix instead defines the logical layout of the custom compressed file header and data.

Not applicable. The system does not register customers or store database records.

Field	Length	Data Type	Description	Is Mandatory
Magic identifier	4 bytes	Character array	Identifies the custom compressed-file format.	Y
Original file size	8 bytes	Number of bytes in the original file.		Y
Frequency entry	16 bytes	Byte + unsigned integer	Stores a byte value and its frequency.	N
Unique byte count	2 bytes	Unsigned integer	Number of frequency entries in the header.	Y
Encoded data	Variable	Binary data	Packed Huffman-coded bytes after the header.	Y
Valid bit count	8 bytes	Unsigned integer	Number of encoded bits that should be decoded.	Y
Output path	Variable	File path	Path selected by the user for the output file.	Y

The application reports original file size, compressed file size, compression ratio, percentage of space saved and operation status.

Compression Report	Decompression Report
Input file path	Compressed file path
Original file size	Recovered file path
Compressed file size	Recovered file size
Compression ratio	Original and recovered size comparison
Space saved	Operation status

Operation status

Error message, if any

Error message, if any

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1	Accept compression input path	User interface	Compression flow	main.cpp	TC-01	STC-01
2	REQ-3	Calculate byte frequencies	Huffman module	Frequency table	Huffman.cpp	TC-02	STC-02
3	REQ-4	Construct Huffman tree	Huffman module	Tree design	Huffman.cpp	TC-03	STC-03
4	REQ-6	Encode and pack bits	Compression module	Bit writer	Compressor.cpp	TC-04	STC-04
5	REQ-8	Write compressed output file	Compression module	Header/file format	Compressor.cpp	TC-05	STC-05
6	REQ-11	Validate compressed metadata	Decompression module	Validation flow	Decompressor.cpp	TC-06	STC-06
7	REQ-12	Reconstruct Huffman tree	Decompression module	Decoder design	Decompressor.cpp	TC-07	STC-07
8	REQ-15	Recover file byte-for-byte	Decompression module	Recovery flow	Decompressor.cpp	TC-08	STC-08
9	REQ-9	Display compression statistics	Statistics module	Report design	main.cpp	TC-09	STC-09
10	SEC-REQ-2	Respect OS file permissions	File interface	Error handling	main.cpp	TC-10	STC-10